

Negotiated Local Plan Repair for Multi-Robot Warehouse Pathfinding

Sreenitya Thatikunta Arpita Deshmukh Prajna Agrawal
B23CS1072 B23CM1007 B23CS1054

1 Problem statement

A team of robots operates in a warehouse whose floor is modelled as a grid of square cells. At each time step, every robot either moves to an adjacent cell (up, down, left or right) or waits. Each robot is assigned **three tasks**, each consisting of picking up an item next to a shelf and delivering it to a delivery station. After its last task, the robot returns to its own parking cell (its *dock*). Two robots may never occupy the same cell at the same time, and two robots may never exchange cells within a single time step.

The robots first compute collision-free paths. While they execute these paths, three kinds of **disruption** can occur:

- a robot **breaks down**, for 5–20 steps or permanently;
- a cell becomes **blocked**, for 10–40 steps or permanently;
- a robot receives an **emergency task**, which takes priority over its other tasks and has a delivery deadline.

After a disruption, the robots must **repair** their plans. The repair is carried out by the affected robots through communication with nearby robots, without re-running the multi-agent planner from scratch, and with as few robots as possible changing their plans.

Environment. The warehouse map measures 18×33 cells. It contains 25 shelf blocks separated by one-cell-wide aisles, 150 pickup cells adjacent to the shelves, 14 delivery stations on the side walls and 58 docks on the top and bottom walls (see Figure 10).

Table 1: Terms used in this report.

Plan	The sequence of cells a robot will occupy, one per time step.
Disruption	A breakdown, a blocked cell or an emergency task that occurs during execution.
Plan repair	Modifying the existing plans after a disruption, without replanning from scratch.
Altered robots	The number of robots whose plan changes because of a single disruption (lower is better).
Total time (SoC)	Sum of costs: the sum, over all robots, of the time step at which each robot completes its tasks (lower is better).
Extra steps	Total time with disruptions minus the total time of the original, undisturbed plan.
Obstacle density	The number of cells blocked during a run, as a percentage of all free cells.
Communication range	Robots can exchange messages only with robots within 6 cells.

2 Methodology

Initial planning

The initial plan is computed with **prioritized planning using Space-Time A***. Robots are planned sequentially, with the robot that has the longest route planned first. Space-Time A* is the A* shortest-path algorithm applied to positions and time together, so a planned path may also include waiting. Each completed plan is recorded in a shared **reservation table**, which marks the cells and moves a robot will use at each time step. Every subsequent robot plans around the existing reservations, which guarantees that the resulting plans are collision-free.

Plan repair

When a disruption occurs, only the robots whose plans have become invalid are repaired, one at a time. Each robot proceeds through a sequence of repair tiers, ordered from the least to the most disruptive for other robots (Figure 1), and stops at the first tier that produces an acceptable plan.

1. **Individual replanning (Tier 1).** The robot replans its own path while treating all other plans as fixed. The new plan is accepted if it adds at most 4 steps. Only one plan changes.
2. **Negotiation with neighbours (Tiers 2–3).** The robot identifies the nearby robots that obstruct its path and asks small groups of them (one, two, then three robots) to replan jointly with it, first within 6 cells and then within 12 cells. A robot that is asked to move but is itself obstructed may in turn ask its own neighbours, up to two levels deep.
3. **Local group replanning (Tier 3b).** For urgent cases only (an emergency robot, or a robot with no feasible path), the robot and up to 8 nearby robots are replanned together.
4. **Detour or wait (Tier 4).** Otherwise, the robot accepts its longer individual detour. If no path exists at all, it waits in place and retries at the next step. After waiting for 10 steps, it extends its search to robots further away.

Acceptance criterion. Each candidate repair is assigned a score:

$$\text{score} = \text{extra steps of all robots involved} + 3 \text{ per additional robot whose plan changes} + 10 \text{ if an emergency deadline is missed}$$

A lower score is better. A joint repair with neighbours is used only if its score is lower than that of individual replanning. The penalty of 3 per additional robot implements the requirement to minimise the number of altered plans: involving another robot is worthwhile only if it saves more than 3 steps.

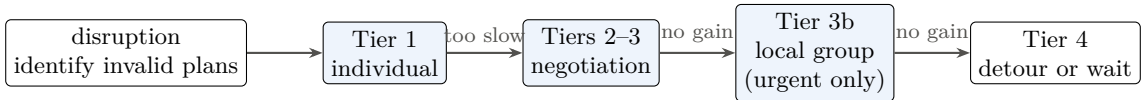


Figure 1: Repair tiers. A robot proceeds to the next tier only when the current tier does not yield an acceptable plan, so other robots are involved only when this reduces the total delay.

Methods compared

- **local:** the proposed method described above.
- **local-flat:** our initial version, without multi-level negotiation, Tier 3b or emergency deadlines.
- **solo:** Tier 1 only, without communication, to measure the benefit of negotiation.
- **full:** replans every robot from scratch after each disruption. Since this violates the requirement of local repair, it serves only as a reference.

Summary. robots are planned sequentially and record their paths in a shared reservation table. After a disruption, each affected robot applies the least disruptive repair first and involves other robots only when doing so reduces the total delay.

3 Experiments and results

Experimental setup. Every configuration was run with **10 random seeds**, and all four methods were evaluated on identical warehouses and disruption sequences. By default, each run uses an obstacle density of 3%, two breakdowns and two emergencies. We varied the **number of robots** (5 to 50) and

the **obstacle density** (0% to 15%), and ran additional experiments with exactly **one disruption** per run to measure the number of plans altered by a single disruption. In total, 1,700 runs were performed. **No collisions occurred in any run**, and all tasks were completed in every run except one **solo** run (Section 4).

Table 2: Comparison of the four methods with 20 and 50 robots (mean of 10 runs, 12 disruptions per run).

robots	method	total time	altered per dis.	emergency on time	repair ms	robots	method	total time	altered per dis.	emergency on time	repair ms
20	local	2612	2.06	65%	126	50	local	6958	5.88	35%	2270
	local-flat	2608	2.11	45%	81		local-flat	6974	5.75	15%	1102
	solo	2622	1.93	30%	12		solo	7089	4.62	15%	67
	full	2611	9.22	100%	41		full	7201	39.33	100%	590

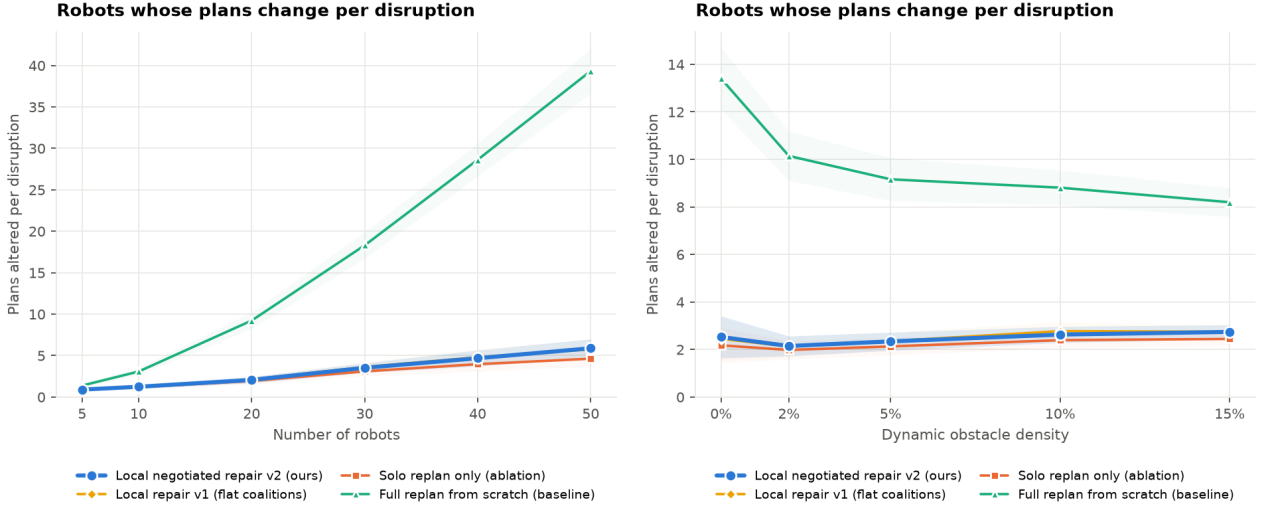


Figure 2: Number of robots whose plans change per disruption as a function of the number of robots (left) and of the obstacle density (right).

Main findings.

1. **Few plans are altered.** The proposed method alters on average about one robot beyond those directly affected by the disruption, whereas replanning from scratch alters between 9 and 39 robots per disruption.
2. **Replanning from scratch is not faster.** With 50 robots, its total time is higher than that of the proposed method (7201 versus 6958 steps), because it reassigns the paths of all robots, including those that were not affected.
3. **Plans altered by a single disruption.** A temporary breakdown alters 2.2 plans (the robots it directly obstructs), a temporary blockage 3.1 and a permanent breakdown 13.5, since the broken robot's tasks are reassigned to other robots. Replanning from scratch alters 22 to 29 plans (Figure 4, left).
4. **The benefit of negotiation grows with fleet size.** With 50 robots, individual replanning (solo) requires 131 more steps than the proposed method.
5. **Improvement over the initial version.** Compared with `local-flat`, the final method reduces total time by 7 to 19 steps with 30 to 50 robots and meets more emergency deadlines (35% versus 15% with 50 robots), while altering a similar number of plans. Its repair time is approximately twice as long.

Effect of the number of robots and the obstacle density

Table 3 reports all configurations, and Figure 3 shows the total time.

Table 3: Results for all configurations (mean of 10 runs). Top: varying number of robots at 3% obstacle density. Bottom: varying obstacle density with 20 robots (disruptions per run in parentheses). Plans altered and repair time are per disruption.

	total time			extra steps		plans altered		repair ms	
	local	solo	full	local	full	local	full	local	full
<i>Number of robots</i>									
5 robots	679	681	679	108	109	0.89	1.38	3	5
10 robots	1330	1330	1332	146	148	1.23	3.07	5	10
20 robots	2612	2622	2611	190	189	2.06	9.22	126	41
30 robots	4093	4148	4131	252	290	3.52	18.29	416	124
40 robots	5510	5572	5617	345	452	4.68	28.62	1115	289
50 robots	6958	7089	7201	377	620	5.88	39.33	2270	590
<i>Obstacle density (disruptions per run)</i>									
0% (4)	2565	2568	2555	143	133	2.52	13.40	70	45
2% (9)	2583	2591	2592	161	171	2.14	10.14	61	45
5% (18)	2630	2652	2629	208	208	2.34	9.16	112	62
10% (31)	2768	2831	2813	347	392	2.63	8.81	247	116
15% (45)	2925	2995	2997	503	575	2.74	8.20	320	1712

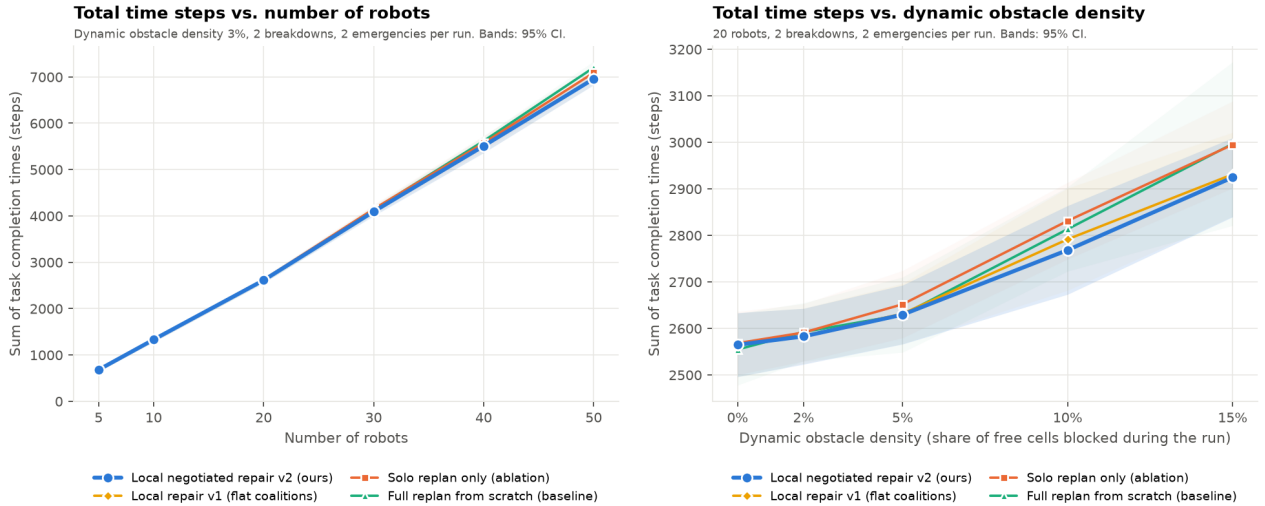


Figure 3: Total time as a function of the number of robots (left) and of the obstacle density (right).

Number of robots.

- Total time increases approximately linearly, by about 140 steps per additional robot, since each robot adds three tasks.
- The time lost to disruptions increases faster, because a more crowded floor offers fewer detours: from 108 to 377 extra steps with the proposed method, compared with up to 620 extra steps when replanning from scratch.
- Up to 20 robots, all methods perform similarly. From 30 robots onwards, the proposed method achieves the lowest total time, requiring 37, 107 and 243 fewer steps than replanning from scratch with 30, 40 and 50 robots respectively.
- The number of altered plans grows slowly with the proposed method (0.9 to 5.9), but almost in proportion to the fleet size when replanning from scratch (1.4 to 39).
- The cost is computation time: repair takes 3 ms with 5 robots and 2.3 s with 50 robots, since more neighbours lead to more candidate groups.

Obstacle density.

- Increasing the density from 0% to 15% increases the number of disruptions per run from 4 to 45 and the total time of the proposed method by 14% (from 2565 to 2925). Each additional disruption costs approximately 9 steps.
- The number of altered plans per disruption remains nearly constant (2.1 to 2.7), because a blocked cell affects only the few robots about to traverse it.
- At low densities, all methods are within about 10 steps of each other. From 10% onwards, the proposed method saves 45 to 72 steps compared with replanning from scratch, and at 15% replanning from scratch becomes considerably slower (1.7 s versus 0.3 s per disruption).

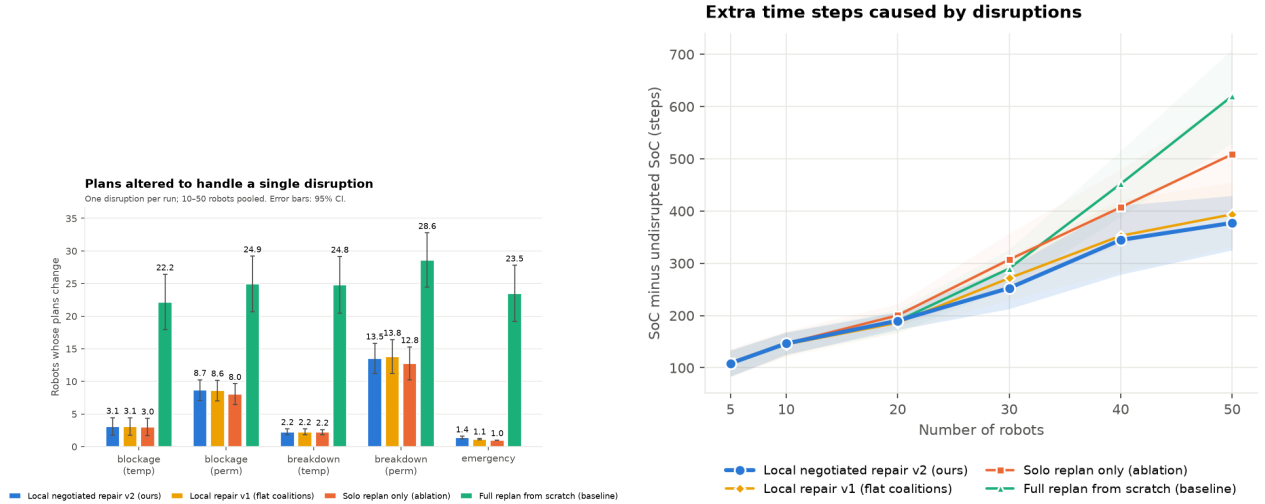


Figure 4: Left: number of robots whose plans change after a single disruption, by disruption type. Right: extra steps caused by disruptions as a function of the number of robots.

Choice of the per-robot penalty. The penalty of 3 per additional robot in the acceptance criterion balances two objectives: altering fewer plans and minimising total time. Figure 5 shows the effect of other values between 0 and 12. Lower values reduce total time slightly but alter more plans, while higher values have the opposite effect. The value 3 was chosen as a compromise.

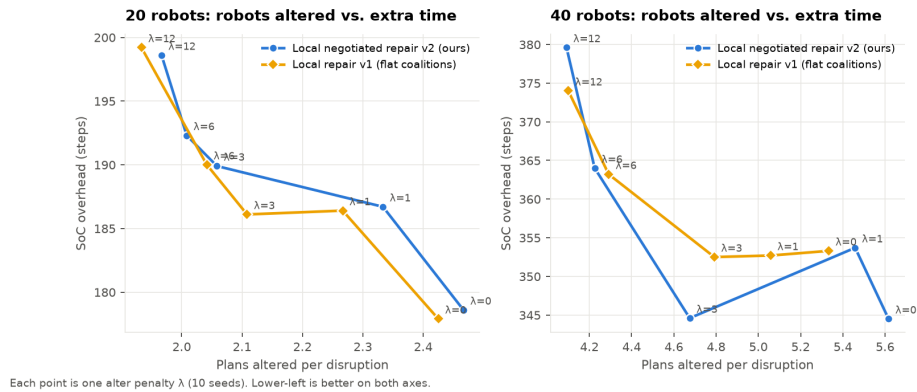


Figure 5: Trade-off between the number of altered plans and the extra steps for different values of the per-robot penalty λ . Points closer to the lower left are better on both measures.

Summary. the proposed method alters only a small number of plans per disruption regardless of the fleet size or obstacle density, and it achieves equal or lower total time than replanning from scratch. Its main cost is computation time, which increases with the number of robots.

4 Settings in which the agents fail

In the main experiments, failures are almost absent. This is partly because the scenario generator never produces unsolvable situations; for example, it never blocks a cell that would make a goal unreachable. To identify the conditions under which the robots fail, we performed a **stress study** of 760 runs in which this safeguard was disabled and the environment was made more demanding. A run is considered to have **failed** if at least one task is never completed. Each robot that becomes stuck is classified as follows:

- **goal unreachable**: its goal can no longer be reached, so the task has become impossible;
- **deadlocked**: its goal is still reachable, but the robot is unable to proceed.

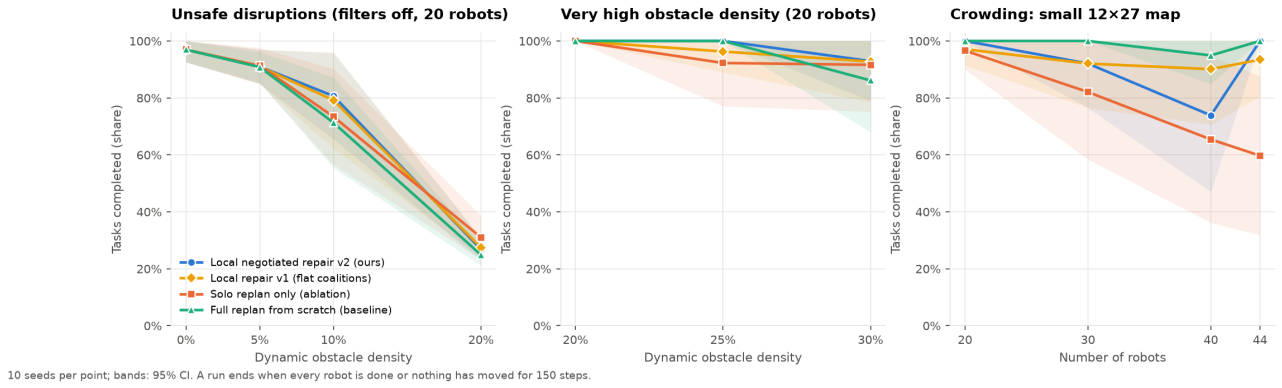


Figure 6: Percentage of tasks completed in each stress setting.

Table 4: Settings in which the robots fail: number of failed runs (out of 10), percentage of tasks completed, and the mean number of stuck robots per run for the proposed method (goal unreachable / deadlocked).

setting	value	local (proposed)			local-flat	full
		failed	completed	stuck	completed	completed
safeguard off, half of the events permanent	density 0%	3/10	96.9%	0.5 / 0.2	96.9%	96.9%
	density 5%	7/10	91.0%	1.5 / 0.3	91.0%	90.8%
	density 10%	10/10	80.6%	2.3 / 3.1	79.2%	71.3%
	density 20%	10/10	26.8%	9.9 / 8.8	27.4%	24.8%
single permanent breakdown, safeguard off	20 robots	2/10	97.5%	0.4 / 0.2	97.5%	97.5%
	40 robots	4/10	96.7%	0.4 / 1.0	96.7%	96.7%
very high density, safeguard on	density 25%	0/10	100%	–	96.3%	100%
	density 30%	1/10	92.9%	0.0 / 2.0	92.7%	86.1%
small crowded map (12 × 27 cells)	30 robots	1/10	92.1%	0.0 / 2.9	92.1%	100%
	40 robots	3/10	73.9%	0.0 / 11.9	90.1%	94.9%
	44 robots	0/10	100%	–	93.4%	100%
reduced communication range (40 robots)	1, 2, 3, 6 cells	0/10	100%	–	100%	100%

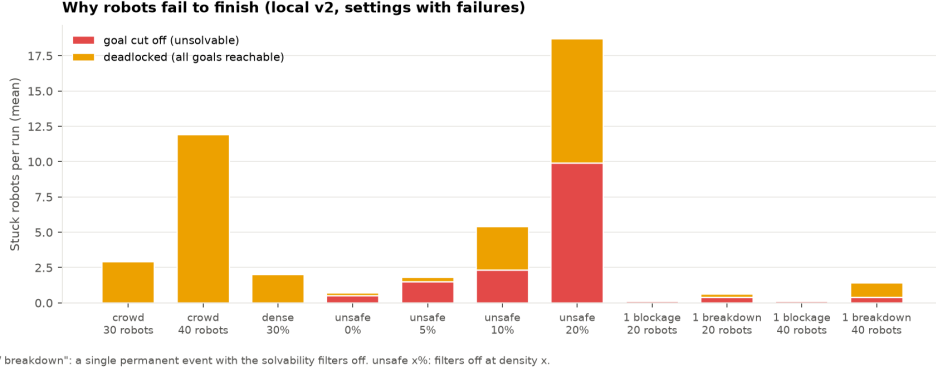


Figure 7: Causes of failure for the proposed method: goal unreachable (red) and deadlock although the goal remains reachable (orange).

1. Unreachable goals. A permanently blocked cell, or a robot that breaks down permanently in a one-cell-wide aisle, can isolate a pickup cell or a delivery station (Figure 8). Such tasks cannot be completed by any method, so all methods fail similarly. Task completion decreases from 97% to 27% as the density increases to 20%. Even a single permanent breakdown causes 2 to 4 out of 10 runs to fail.

2. Secondary deadlocks. A robot whose goal has become unreachable does not abandon its task. It remains in its cell indefinitely, and this cell may be the only passage for other robots. At 10% density, 2.3 robots per run have an unreachable goal, while a further 3.1 robots become deadlocked behind them. More than half of the stuck robots are therefore caused by the robots' response rather than by the impossibility of the task itself. The proposed method is nevertheless more robust than replanning from scratch in this setting (81% versus 71% of tasks completed).

3. Propagation of waiting on crowded maps (main weakness). On the small map with 40 robots, 3 of 10 runs fail with nearly the entire fleet stuck. In one case, a temporary breakdown obstructs 6 robots, which begin to wait. Robots whose plans pass through the cell of a waiting robot then also wait, until all 40 robots are waiting for one another and only 1 of 122 tasks is completed. The repair groups (at most 4 or 8 robots) are too small to resolve this situation, whereas replanning from scratch resolves it. The effect depends strongly on the specific instance: with 44 robots, no run failed.

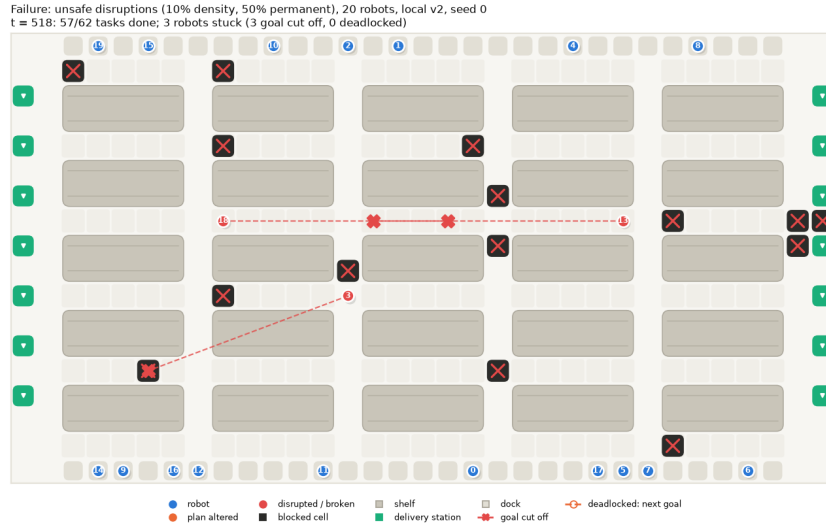


Figure 8: Example of failure: permanently blocked cells (black) make the goals of robots 3, 13 and 18 (red) unreachable, and 5 of 62 tasks are never completed.

4. Other settings.

- **Very high obstacle density:** at 30% density, with all tasks still solvable, 1 of 10 runs results in a deadlock.

- **No communication:** without negotiation (**solo**), all 49 remaining robots became stuck after a single permanent breakdown with 50 robots (Figure 9), and 5 of 10 runs failed on the small crowded map. The proposed method completed both.
- **Emergency deadlines:** in crowded warehouses, only 5–35% of emergency deliveries meet their deadline, although all of them are eventually completed.

The robots did **not** fail with a reduced communication range (a range of 1 cell increases total time by only 1%, because a robot that has waited for 10 steps extends its search), nor in any solvable configuration with up to 50 robots and 20% obstacle density.

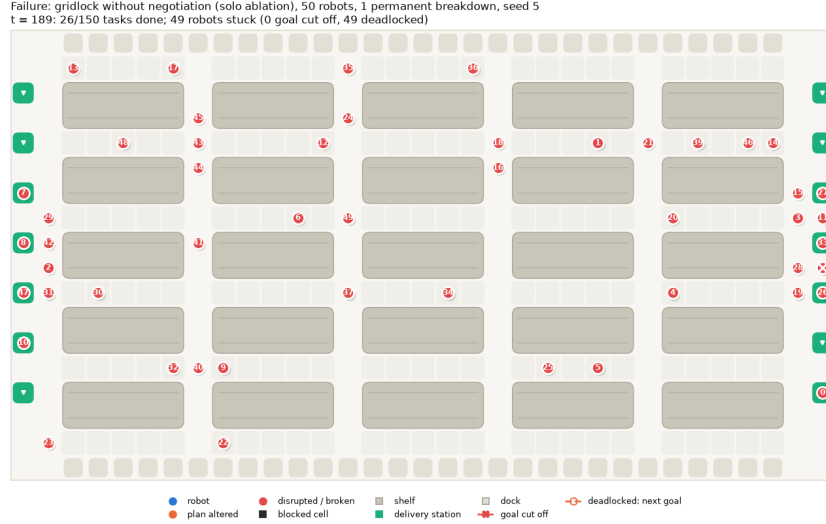


Figure 9: Failure without communication (**solo**): after a single permanent breakdown, all 49 remaining robots (red) wait for one another indefinitely. The proposed method completes all 150 tasks in the same situation.

Summary. the robots fail when (1) a goal becomes unreachable, (2) a stuck robot obstructs others, (3) waiting propagates through a highly crowded map, (4) the obstacle density is very high, or (5) the robots cannot communicate.

5 Graphical demonstration

The simulations were visualised with the Python library matplotlib. We produced an MP4 video of 20 robots handling 12 disruptions (Figure 10 shows one frame) and videos of the failure cases described in Section 4. For a few steps after each disruption, the robots whose plans changed are shown in orange, together with their new paths.

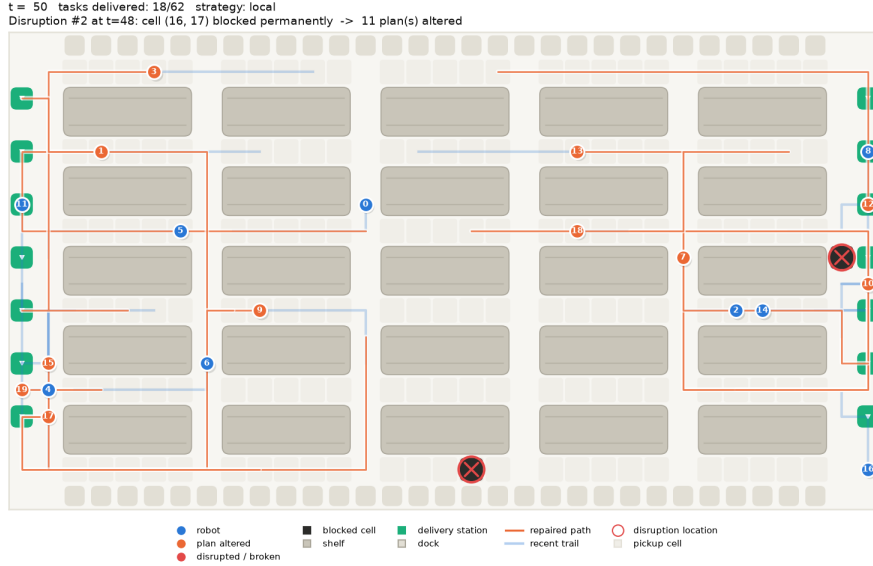


Figure 10: A frame from the demonstration video, shortly after a cell was permanently blocked (red circle). The 11 robots whose plans changed are shown in orange, with their new paths.

6 Limitations

- **Incompleteness of prioritized planning:** sequential planning may fail to find a plan even when one exists.
- **No task abandonment:** a robot whose goal becomes unreachable waits indefinitely and may obstruct other robots.
- **Propagation of waiting:** on crowded maps, waiting can spread across the fleet; replanning a larger region when many robots are waiting would address this.
- **Soft emergency deadlines:** a missed deadline only adds a penalty, so most deadlines are missed in crowded warehouses.
- **Computation time:** repair takes about 2 seconds per disruption with 50 robots in Python.
- **Idealised communication:** messages are assumed to be delivered instantly and without loss.